

① Databases

- Query evaluates tuple by tuple, Only one tuple at a time
- If we want to compare two or more tuple of same table or diff. table then we need to combine those tuples into a single tuple using join.
- RA will always produce distinct tuple {No duplicate tuple}

① π → project column data
 $\pi_{\langle \text{columns} \rangle}(R)$
 not obey commutative property

② σ → select tuples based on predefined condⁿ
 $\sigma_{\langle \text{selection cond}^n \rangle}(R)$
 $\sigma_{A \cap B}(R) = \sigma_B(\sigma_A(R))$ obey com $\sigma_B(\sigma_A(R)) = \sigma_A(\sigma_B(R))$

③ Cross product (x)
 $R \times S$
 all attr of R followed by all attr of S.

$\pi_{S.\text{Sname}}(\sigma_{(R.\text{Cid}=C1 \wedge R.\text{Sid}=C.\text{Cid})}(R \times S))$

To use set theory operations on any two relⁿ, those relⁿ must be Union compatible

1. Same degree (same no. of attr)
2. Same domain of each corresponding attribute

Note: After set opⁿ resulting relⁿ will take names of attribute from LHS relⁿ

④ Union ($A \cup B$) → contains all tuples from both relⁿ

⑤ $A - B$ → tuples in A but are not present in B

⑥ $A \cap B = A - (A - B)$ $R \cap S = R - (R - S)$

⑦ Rename (ρ) → rename attribute, name of relⁿ on both

$\rho_{\text{FinalYrStudents}}(\text{Students})$
 new name for relⁿ

$\rho_{(S10, Sname)}(\text{Students})$

$\rho_{\text{FinalYrStudents}}(S10, Sname)(\text{Students})$

* if want to change just 1 attribute still have to write all attributes of Relation

Types of join

1) Inner join { only select tuples from both relⁿ that satisfy join condⁿ }

- a) Theta join
- b) Equi join
- c) Natural join

2) Outer Join { satisfy that did not satisfy the join condⁿ }

- a) left outer join
- b) Right outer join
- c) Full outer join

a) Theta Join → any condⁿ used in Selection Criteria
 $\{ >, <, \geq, \leq, =, \neq \}$
 not equal

$R_1 \bowtie_{\theta} R_2$
 can be any condⁿ

Student $\bowtie_{(Student.Sid < Emp.Sid)}$ Employee = $\pi_{\sigma_{Student.Sid < Emp.Sid}}(Student \times Emp)$
 Join condⁿ " θ "

b) Equi join → Theta join having always equality condⁿ
 need not be on same or common attribute

Student $\bowtie_{(Student.Sid = Employee.Eid)}$ Employee

$\bowtie_{Student.Sid = Enroll.Sid}$

Note:- In theta join or Equijoin, we need to specify the join condⁿ Explicitly, ② which is not that case with Natural Join

In Natural Join ① we don't need to specify the join condⁿ & ② join condⁿ is always Equality condⁿ on all common attributes

on all common attributes relations. [name & type must be same]

Additionally, a natural join removes the duplicate columns involved in the equality comparison so only one of each compared column remains

Student (Sid, Sname) & Enroll (Stud-id, Cid)
No common attribute

$R \bowtie S \rightarrow$ Step 1: Obtain $R \times S$

Step 2: From $R \times S$, select tuples based on equality condⁿ on all common attr. of R & S

Step 3: From Step 2 op, Project distinct attr. of R & S

① $R(A, B, C), S(A, D, E)$

$$R \bowtie S = \pi_{A, B, C, D, E} (\sigma_{R.A=S.A} (R \times S))$$

② $R(A, B, C) \& S(B, C, D)$

$$R \bowtie S = \pi_{A, B, C, D} (\sigma_{\substack{R.B=S.B \\ R.C=S.C}} (R \times S))$$

③ $R(A, B, C) \& S(D, E)$

$$R \bowtie S = \pi_{A, B, C, D, E} (R \times S) = R \times S \quad \text{If no common attr., then o/p of natural join will be same as cross prod}$$

Student		Enroll		Student $\bowtie$ Enroll			Equality cond ⁿ
Sid	Sname	Sid	Cid	Sid	Sname	Cid	
2	A	2	C ₁	2	A	C ₁	Equality cond ⁿ
4	B	2	C ₂	2	A	C ₂	
3	A	3	C ₂	3	A	C ₂	

Outer Join $\rightarrow$ along with tuple that satisfy matching criteria, we also include tuples that do not match criteria, either from LHS or RHS or both

① Left Outer Join ($\ltimes$) $\rightarrow R \ltimes S$ All tuples of $R \bowtie S$, tuples from LHS relⁿ (R) that failed the con

R			S			R $\ltimes$ S			
A	B	C	B	C	D	A	B	C	D
1	2	3	2	3	4	1	2	3	4
1	4	5	3	4	5	1	4	5	NULL

tuples from $R \bowtie S$
 $\rightarrow$ LHS R tuple

② Right Outer Join ($\rtimes$)

③ Full ($\ltimes$)

① Division (÷) → used whenever query is not every on all

③

Enroll (E)

Sid	Cid	fee
S ₁	C ₁	
S ₂	C ₁	
S ₁	C ₂	
S ₂	C ₂	
S ₁	C ₃	
S ₁	C ₃	

Query: Retrieve Sids of Students who have enrolled for all courses

$$\pi_{sid, cid} (Enroll) \div \pi_{cid} (courses)$$

or S₁

C

Cid	Cname
C ₁	OS
C ₂	DM
C ₃	DBMS

did not enroll for all courses

Sid of student who

S ₁	C ₁
S ₂	C ₁
S ₁	C ₂
S ₂	C ₂
S ₁	C ₃
S ₂	C ₃

when every student enrolled for every course

In this table we will get info that which student did not enroll for all courses

[Student who enrolled for proper subset of all courses]

$$\pi_{sid} (E) - \pi_{sid} \left[\pi_{sid} (E) \times \pi_{cid} (C) - \pi_{sid, cid} (E) \right] = \pi_{sid, cid} (E) \div \pi_{cid} (C)$$

② Consider the following relational tables:

Supplier (Sid, Sname, Rating)

Parts (Pid, Pname, Color)

Catalog (Sid, Pid, Cost)

i) Sid of all suppliers
 $\pi_{sid} (Supplier)$

ii) who supplied some parts
 $\pi_{sid} (Catalog)$

iii) Sid of supplier whose rating is more than 5
 $\pi_{sid} [\sigma_{Rating > 5} (Supplier)]$

iv) Sid of supplier who supplied some red color part

$$\pi_{sid} \left[\sigma_{\substack{P.pid = C.pid \\ C.pid = P.pid \\ P.color = 'red'}} (P \times S) \right]$$

v) Some red on some green

$$a) \pi_{sid} \left[\sigma_{\substack{P.pid = C.pid \\ P.color = 'red' \vee P.color = 'green'}} (P \times S) \right]$$

BOTH CORR

$$a) \pi_{sid} \left[\sigma_{\substack{P.pid = C.pid \\ P.color = 'red'}} (P \times S) \right] \cup \pi_{sid} \left[\sigma_{\substack{P.pid = C.pid \\ P.color = 'green'}} (P \times S) \right]$$

b) $\pi_{sid} [\sigma_{color = red} (CMP)]$

$$b) \pi_{sid} [\sigma_{\substack{color = red \\ color = gr}} (CMP)]$$

$$b) \pi_{sid} [\sigma_{color = red} (CMP)] \cup \pi_{sid} [\sigma_{color = green} (CMP)]$$

c) $\pi_{sid} [C \bowtie \sigma_{color = red} (P)]$
most eff. query

$$c) \pi_{sid} [C \bowtie \sigma_{color = red} (P)] \bowtie \pi_{sid} [C \bowtie \sigma_{color = green} (P)]$$

$$\pi_{C_1, sid} \left[\begin{array}{l} \sigma_{(C_1, sid = C_2, sid)} \left[(C_1 \times P_1) \wedge (C_2 \times P_2) \right] \\ (C_1, pid = P_1, pid \wedge P_1, color = 'red') \vee (C_2, pid = P_2, pid \wedge P_2, color = 'Green') \end{array} \right] \quad (2)$$

or
C₂ color
not eff but if given in Green as option

Q Red & Green color partp.

$$i) \pi_{C, sid} \left\{ \sigma_{(C, pid = P, pid \wedge P, color = 'red' \vee P, color = 'Green')}(C \times P) \right\}$$

ii)
iii)

Color can't be Red as well as Green in the same topic
∴ op will be empty.

correct

$$(i) \pi_{C, sid} \left[\sigma_{C, pid = P_1, pid \wedge P_1, color = 'Red'}(C \times P_1) \right] \cap \pi_{C, sid} \left[\sigma_{C, pid = P_2, pid \wedge P_2, color = 'Green'}(C \times P_2) \right]$$

$$ii) \pi_{sid} \left[\sigma_{color = 'Red'}(C \times P) \right] \cap \pi_{sid} \left[\sigma_{color = 'Green'}(C \times P) \right]$$

$$(iii) \pi_{sid} (C \bowtie \sigma_{color = 'red'}(P)) \cap \pi_{sid} (C \bowtie \sigma_{color = 'Green'}(P))$$

$$\pi_{C, sid} \left[\begin{array}{l} \sigma_{(C_1, sid = C_2, sid)} \left[(C_1 \times P_1) \wedge (C_2 \times P_2) \right] \\ (C_1, pid = P_2, pid \wedge P_1, color = 'red') \wedge (C_2, pid = P_2, pid \wedge P_2, color = 'Green') \end{array} \right]$$

Q Supplied all partp

$$\pi_{sid, pid} (Catalog) \div \pi_{pid} (Parts)$$

$$\pi_{sid}(C) - \pi_{sid} \left[\pi_{sid}^{(i)} \times \pi_{pid}(P) - \pi_{sid, pid}(C) \right]$$

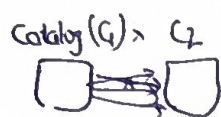
Q all red color parts supplied

$$\pi_{sid, pid} (Catalog) \div \pi_{pid} (\sigma_{color = 'Red'}(P))$$

$$\pi_{sid}(C) - \pi_{sid} \left[\pi_{sid} \times \pi_{sid} [\sigma_{color = 'red'}(P)] - \pi_{sid, pid}(C) \right]$$

5

Q atleast two parts



$$\pi_{C1.sid} \left[\sigma_{\substack{C1.sid = C2.sid \\ C1.pid \neq C2.pid}} (C1 \times C2) \right]$$

If we do not use this and then all sid of catalog table will be present in output, (Each tuple will combine with itself)

Q exactly one ~~two~~ parts

Suppliers who have supplied exactly one part

= Atleast one part

- Atleast 2 parts

$$= \pi_{sid}(C) - \pi_{sid} \left[\sigma_{\substack{C1.sid = C2.sid \\ C1.pid \neq C2.pid}} (C1 \times C2) \right]$$

Q Atmost one part

= 0/more

- 2/more

$$= \pi_{sid}(Suppliers) - \pi_{sid} \left(\sigma_{\substack{C1.sid = C2.sid \\ C1.pid \neq C2.pid}} (C1 \times C2) \right)$$

Q atleast three parts

$$\pi_{sid} \left[\sigma_{\substack{(C1.sid = C2.sid) \wedge (C2.sid = C3.sid) \wedge (C1.sid = C3.sid) \\ (C1.pid \neq C2.pid) \wedge (C2.pid \neq C3.pid) \\ (C1.pid \neq C3.pid) \\ \text{it is mandatory}}} (C1 \times C2 \times C3) \right]$$

Q most expensive parts

$$\pi_{sid} \left[\left(\text{All tuples of Catalog table} \right) - \left(\text{Tuples of the catalog table in which cost is not max.} \right) \right]$$

↳ cost is max. tuple

$$\pi_{sid} \left\{ (Catalog) - \pi_{C1.sid, C1.pid, C1.cost} \left(\sigma_{C1.cost \leq C2.cost} (C1 \times C2) \right) \right\}$$

Sid & Product wise check

Can't just use c1.sid only

eg

S ₁	P ₁	20
S ₃	P ₂	20
S ₃	P ₃	10

then S₃ will cut